The OCUDU dApp Platform: An Open Runtime and E3 Interface for Real-Time AI-RAN

Timothy O'Shea, Matthew Pennybacker, Andriy Kharchenko
DeepSig Inc., Arlington, VA, USA

***Abstract*—The band below 10 ms in a 5G NR distributed unit (DU), where link adaptation, per-slot scheduling, channel estimation, and the receiver itself run, is where machine learning has shown its largest gains and where no open platform has let anyone run it. Prior dApp frameworks reached the band as external observers of an export stream. This paper is a guided introduction to the OCUDU dApp platform: an open runtime and E3 interface under which independently built, signed AI-RAN applications execute inside a production DU under 3 timing contracts, resident on the GPU receive chain (Class A), inside the scheduler's 100 μs admitted deadline (Class B), or as never-blocking observers whose results the scheduler consumes (Class C), with the conventional path always armed and every authority typed, validated, and operator-bounded. The paper explains how the runtime, the embedded E3 agent, and the 3 public repositories fit together, where each mechanism lives in the code, how a dApp is built, signed, admitted, and driven through its lifecycle, and how the same surface serves a Python script, an operator's console, and an LLM agent. On a GB10 gNB with attached handsets, 4 dApps across all 3 classes run at once (260,000+ neural-equalizer invocations, 0 fallbacks) and equalizer variants are compared over the air by lifecycle operations alone. Measured checkpoints are reported with their conditions. Platform, SDK, and a zero-hardware quickstart are public under BSD-3-Clause-Clear as a preview release of the OCUDU AI-RAN Working Group 2, inviting feedback, new use cases, and independent vetting ahead of upstreaming into the OCUDU mainline.**

***Index Terms*—AI-RAN, dApp, O-RAN, E3, 5G NR, DU, GPU, real-time control, ISAC, spectrum sensing, neural receiver, MLOps**

I. Introduction

The O-RAN control architecture leaves its most valuable real estate unmanaged. rApps operate at 1 s and above; xApps reach down to roughly 10 ms [1], [2]. Everything faster, link adaptation, per-slot scheduling, channel estimation, spectrum reaction inside a coherence time, has been sealed inside the vendor's DU. Yet this band is where learned components have shown the largest gains, from neural receivers [3] to learned link adaptation, and where sensing applications such as integrated sensing and communication (ISAC) must live to see the channel at all.

The dApp concept [4], given its E3 interface (the real-time interface between a dApp and its host RAN node) and a working OpenAirInterface framework by Lacava et al. [5]–[7], and echoed by NVIDIA's Aerial dApp container [8]–[10], named this tier and proved the demand: the O-RAN research arm has published dApp use cases [11], vendors are opening their lower PHY [12], and deployment characterizations are appearing [13]. These systems share 1 structural ceiling, shown hatched in Fig. 1: the dApp is an *external observer loop*. L1 exports IQ or KPIs, a separate process consumes them, and control, where it exists, returns as a coarse indication with no admission bound, no deadline, and no fallback. The highest-value applications, a neural receiver on the L1's own tensors or a learned policy that answers before the scheduling decision commits, cannot be expressed in that shape.

This paper introduces the OCUDU dApp platform, which opens the band as a managed, open runtime, and it is written as a guide to the system as built. The thesis behind the platform is that AI-RAN's upside will be realized quickly only if this tier is opened to a diverse community of vendors, operators, and researchers, and that 1 infrastructure can carry all of it: the same package format, E3 plane, and safety machinery that let a Python script watch CRC verdicts also let a neural receiver replace the receive chain, let a sensing worker steer the scheduler, close a continual-learning loop from live capture to model rollback, and let several vendors ship dApps that 1 operator composes at runtime. The paper is written as an architecture overview and tutorial: it proceeds from the architecture at a glance (Section III), through the 3 execution classes (Section IV) and the E3 plane (Section V), to where the code lives (Section VI), how to work with it (Section VII), the applications it carries (Section VIII), the vendor ecosystem (Section IX), the latency and safety architecture (Section X), and the measured evidence with its limits (Section XI).

II. Prior dApp Frameworks and Their Boundary

The reference dApp implementation [5] pairs a Python authoring library with *libe3* [7], a patched OpenAirInterface gNB exporting IQ through an E3 service model, and a FlexRIC extension for xApp–dApp coordination; its canonical dApp senses the spectrum on exported IQ and returns a PRB blacklist that the MAC applies. NVIDIA's container [8], [9] pairs an E3 manager with a Triton inference server beside the CUDA L1, publishing RAN data through shared-memory descriptors to GPU inference backends, applied to real-time uplink interference detection in InterfO-RAN [14]. Both are genuine sub-RIC control and both validated E3 as the interface.

Three consequences follow from the external-loop shape regardless of implementation quality. *First*, inline applications

*Preview version. This paper accompanies a preview release of the platform and will receive updates as the working group's review, new use cases, and measurements land.

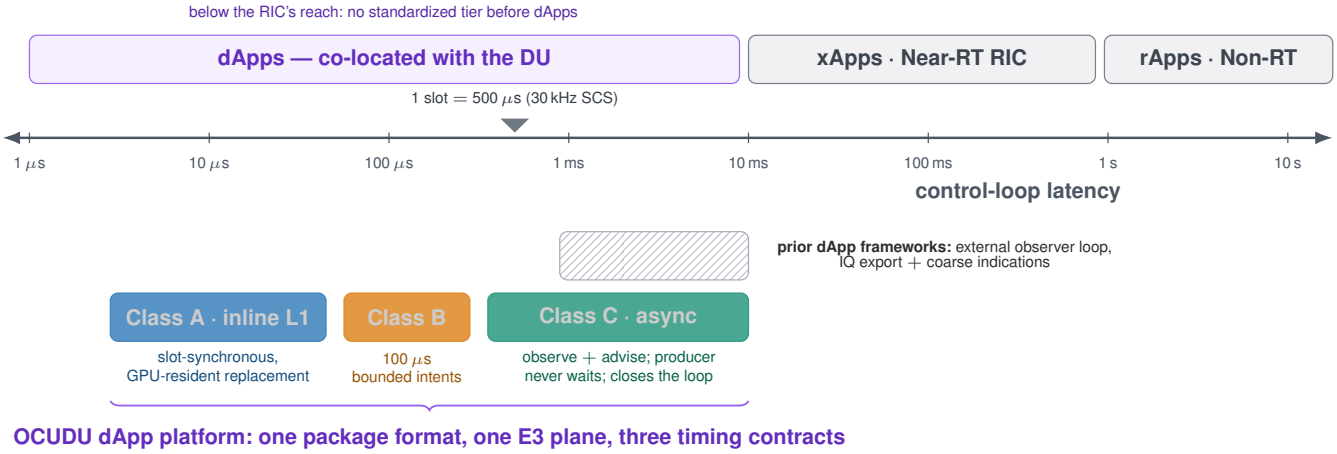


Fig. 1. The control-loop ladder. O-RAN's RICs stop at about 10 ms. Prior dApp frameworks entered the sub-10 ms band only as external observer loops (hatched). OCUDU covers the whole band with 3 timing contracts under 1 package format and 1 E3 plane.

cannot be expressed: a neural receiver must run on the L1's tensors and stream inside the slot, and an external process pays serialization, wakeup, and CUDA-context costs to reach data the PHY already holds [15]. *Second*, the radio inherits the dApp's failure modes: without producer-side isolation, admission control, deadlines, and an always-armed conventional path, an experimental component can degrade or stall the cell. *Third*, operations do not scale: without signed packaging, typed configuration, model lifecycle, and uniform observability, every dApp is a bespoke deployment, and the machinery an operator needs to compose dApps from several vendors on 1 cell had not been built. The platform addresses each while keeping the observer pattern fully supported as its entry point (Class C over E3), so existing dApp-style consumers port directly.

III. The Platform at a Glance

OCUDU, the Linux Foundation's open-source 5G CU/DU project [16], is a production-grade 5G NR stack whose GPU-resident CUDA L1 and fronthaul acceleration are described in [15]. The dApp runtime adds 3 seams to that stack and 1 management plane over them (Fig. 2):

- **Class A, Resident Inline L1.** Three interfaces on the PUSCH receive chain, channel estimation (`0x00010003`), estimation plus equalization (`0x00010004`), and the full receiver to soft LLRs (`0x00010005`), invoked in process on the lane's CUDA stream over GPU-resident tensors.
- **Class B, Bounded Real-Time Control.** One interface at the MAC scheduler's decision boundary (`0x00010006`), a direct call that returns typed intents under a 100 μs admitted deadline.
- **Class C, Asynchronous Observation and Advisory.** Two interfaces over the products the PHY already computes, the full uplink spectrum grid (`0x00010001`) and the SRS channel estimates (`0x00010002`), consumed without ever blocking the producer, in process, in a supervised worker process, or as a portable E3 client anywhere.
- **The E3 plane.** One embedded agent, published ASN.1 and FlatBuffers schemas, 3 carriers, 8 telemetry streams, and 1 management surface for lifecycle, configuration, models, metrics, incidents, health, reservations, and RAN control; an MCP server in the SDK exposes the same KPIs and knobs to LLM agents as tools.

A dApp is a shared object built out of tree against the installed SDK, described by a signed manifest and an SPDX software bill of materials (SBOM), staged into a catalog named in the gNB YAML, and driven through an explicit lifecycle (`load` → `prepare` → `arm` → `shadow` → `activate`) over E3. The third-party boundary is a frozen, size-tagged C ABI with a checked-in layout fingerprint verified across GCC and Clang, C11 and C++17, loaded with `dlopen` [17]. dApps never call each other; cooperation flows only through host-owned, schema-versioned caches. The contracts are backend-neutral, a package declares a CPU (x86, Arm) or CUDA backend and receives matching resident hooks, but the emphasis is GPU-first: the compute headroom beside the baseband is what lets AI-RAN applications be tried at full fidelity, then optimized, without displacing the radio's own budget. Four invariants hold everywhere: the host owns memory, streams, validation, and fallback; the conventional path is always armed; every authority is typed, validated field by field, and operator-bounded; and dApps never call each other.

IV. The Three Execution Classes

A. Class A: resident inline L1

The host supplies GPU-resident tensors (grid views, DM-RS pilots, compact data-RE indices) and its own per-lane CUDA stream [18], [19]; it checks the grant shape against the module's declared admission profile with a fixed sequence of integer bounds before any module code runs, and the module enqueues kernels and returns without allocating or

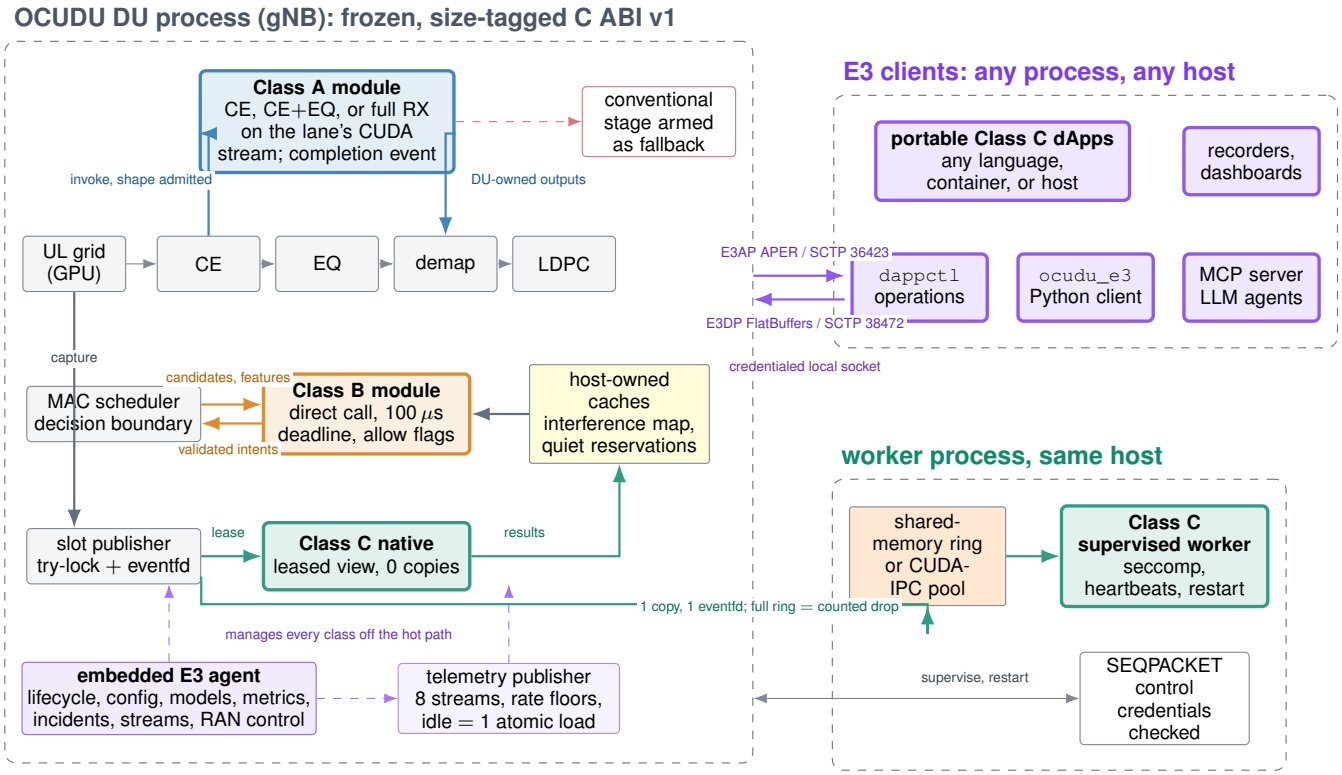


Fig. 2. The platform as implemented. Inside the DU process: Class A modules are invoked on the PUSCH lane's own CUDA stream and write DU-owned outputs; a Class B module is a direct call at the scheduler's decision boundary returning validated intents; native Class C modules take leased views of each captured slot and publish results into host-owned caches the scheduler reads back. Outside: supervised Class C workers behind a shared-memory ring or CUDA-IPC pool, and E3 clients of every kind. One embedded E3 agent manages all of it off the real-time path.

synchronizing. Completion is measured on a separate completion clock by per-lane CUDA events: a late completion is an observed incident that feeds the lane breaker, never a replayed result, while the conventional stage runs as same-invocation fallback whenever the module bypasses, fails, or is disabled. The 3 depths rejoin the chain at successively deeper points, after channel estimation, after equalization, or at the pre-descrambling LLRs; no hook replaces equalization alone, because the equalizer interface also implements channel estimation, and each depth bounds authority to 1 segment whose tensors already exist in GPU memory. Model management is first class: a bounded stage, validate, warm, activate, rollback slot machine swaps weights on a live cell under generation checks, with no allocation on the hot path.

B. Class B: bounded real-time control

The MAC scheduler invokes an admitted Class B module at its decision boundary with candidate UEs, masks, and a feature block (including post-equalization noise handed forward from Class A and the interference context published by Class C); the module returns intents, priority reordering, PRB avoidance, MCS and TPC hints, that the host validates field by field against operator *allow flags* before committing. The complete boundary, from feature construction through commit, runs under 1 admitted deadline of 100 μs. A late or invalid answer is discarded and the conventional priorities, always computed first, stand. The reference scheduler is deliberately a baseline replica, so enabling it changes nothing until the operator grants authority flag by flag, evaluation mode first.

C. Class C: asynchronous observation and advisory

Class C is where prior architectures ended and where OCUDU deliberately begins. Full uplink spectrum grids and SRS channel estimates are consumed at 3 isolation levels: native in-process modules over leased zero-copy views or route-owned snapshots; supervised worker processes over a shared-memory ring or, for CUDA workers, a bounded CUDA-IPC export pool filled by 1 device-to-device snapshot that never maps live L1 memory; and portable E3 clients on any host. The producer never waits: publication is a try-lock plus a non-blocking eventfd, 1 `write` and 2 `read` syscalls, and a full ring or busy consumer is a counted drop. Class C results flow *back*: an admitted spectrum authority publishes per-PRB interference maps and quiet-period intents over the authenticated control channel; the host validates provenance and capability, grants reservations under policy, and the scheduler's avoidance mask consumes both. Fig. 3 places all 3 contracts on 1 slot.

D. A continuum of performance and trust

The classes are points on a performance–trust continuum that the design spans on purpose (Fig. 4). A resident Class A

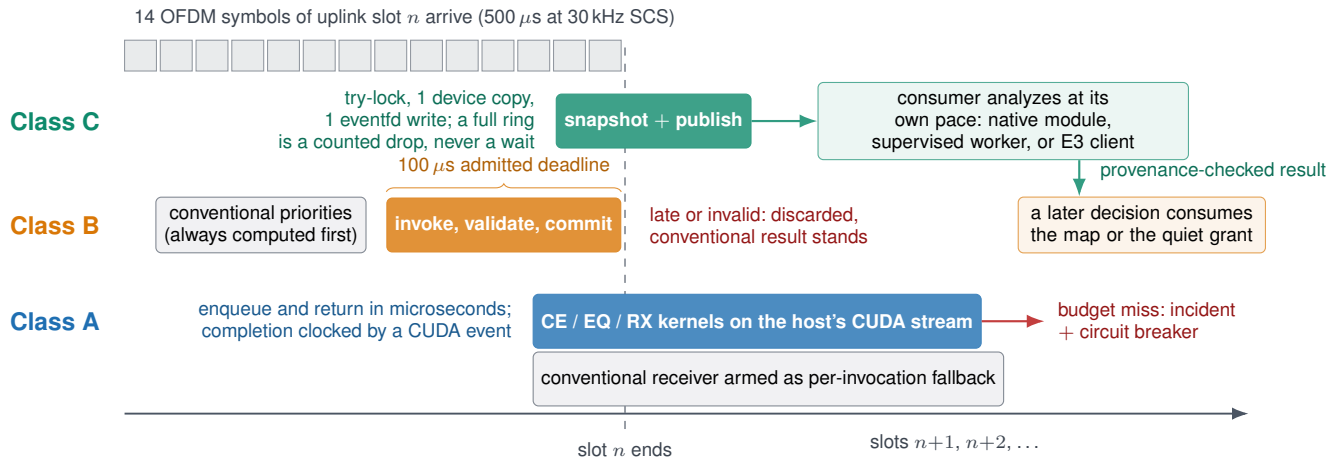


Fig. 3. One uplink slot across the 3 timing contracts. Class B decides early inside a bounded boundary; Class A replaces receive stages inline as the slot completes, measured on the completion clock; Class C forks a snapshot without blocking and closes the loop at a later decision through provenance-checked results.

module runs in process on the host's stream with zero-copy tensors and microsecond authority, and must therefore be the most trusted artifact in the system: signed native code in the DU's address space. An E3 client at the other end runs in any process, language, or host, needs almost no trust, and pays for that isolation in latency, copies, and rate floors. Between them sit Class B's bounded in-process boundary, native Class C placement, and the supervised process with its 1-copy export and seccomp containment. The choice belongs to the adopter: a vendor or integrator enters at the placement that matches its containment requirements and maturity, typically the isolated end, and moves inward as shadow runs, A/B evidence, and incident-free operation accumulate. Placement is a Class C choice; Class A and authoritative Class B are in process by definition, and a package declaring an out-of-process inline hook is rejected at catalog time. Section XI reports what each position costs on the reference host, and the companion interface study [20] measures every position against the mechanisms of the prior frameworks.

V. The E3 Plane

Everything is controlled and observed over E3 (Fig. 5) through published, versioned schemas: an ASN.1 management service model and E3AP envelope encoded as APER, a telemetry service model, and a FlatBuffers data profile. Three carriers serve them: a credentialed local UNIX socket for same-host tooling, E3AP over SCTP (port 36423) with message-boundary framing and bounded fragmented indications, and the data profile over SCTP (port 38472) for poll and lease consumption compatible with the reference clients. The APER decode path is defensively bounded end to end (constraint checks before field reads, bounded reassembly, size-capped strings), so a malformed peer cannot destabilize the agent; transport authentication is deliberately outside the protocol in this release, and the SCTP carriers bind to loopback by default (Section XI). E3 here is a real-time local evolution of E2's setup, subscription, indication, and control philosophy, bridgeable to E2 but not a ratified O-RAN interface; APER is the normative encoding, and the `libe3` adapter is the compatibility path to standard E3AP [2], [7]. SCTP is the carrier E2 standardized and the one several large vendors prefer for RAN control planes, which is why it is a first-class carrier here and not an afterthought.

Telemetry (Table I) differs from prior exports in 2 properties. *Rate discipline*: per-RNTI floors, per-subscription thinning, and a non-zero-by-default per-stream hard floor are enforced before any capture work, and an idle stream costs 1 atomic load per event, so subscribing a recorder cannot destabilize the cell. *Completeness*: `pusch.grid` carries raw pre-equalization resource elements with every parameter needed to re-run the receive chain (HARQ and RV, scrambling, LDPC base graph, MCS, DM-RS layout, DC position, UCI), the substrate for replay-verified datasets.

Management is 1 surface: inventory; asynchronous lifecycle jobs; typed configuration against per-package schemas; model staging with validation, warm-up, atomic activation, and rollback; metrics and metric schemas; incidents; health; quiet-reservation queries; served-cell information; and a bounded set of RAN controls (UE release to idle, handover, RU gain). Three monotonic generations, inventory, instance, and configuration, make concurrent control safe through compare-and-swap, and every mutation supports `dryRun`. The same operations work identically over the local socket and SCTP, from the C++ `dappctl`, the Python client, or the MCP server of Section VIII.

VI. Where the Code Lives

The platform is published as 3 repositories under the OCUDU AI-RAN Working Group 2 (Fig. 6), cloned side by side with the platform as `ocudu/` and the quickstart as `ocudu-docker/`. These repositories are where the working group's development is ongoing: the current state is a preview

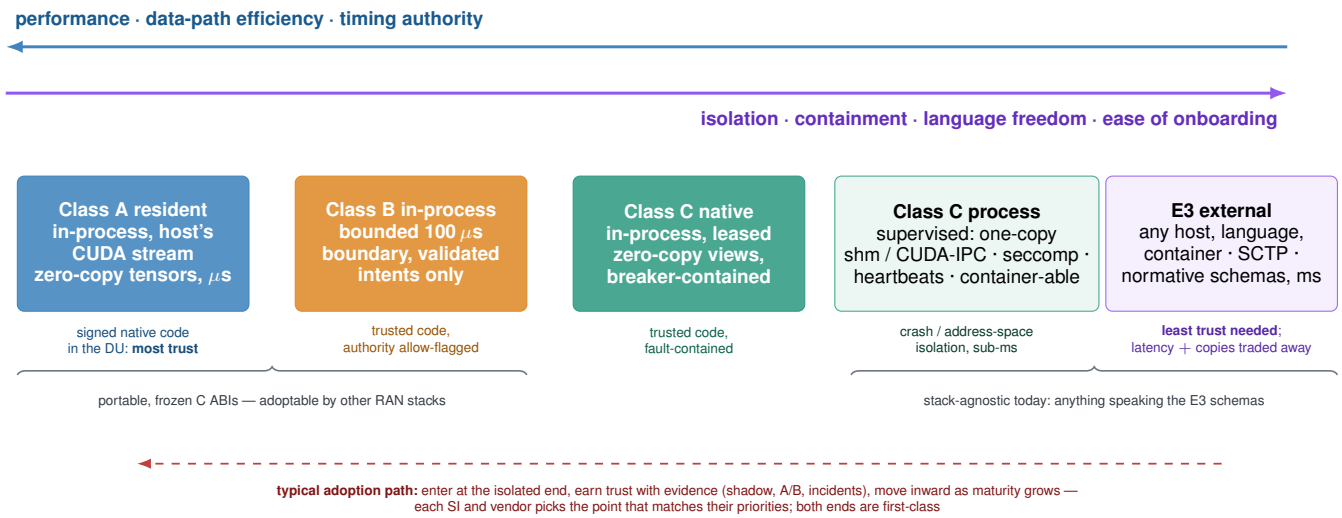


Fig. 4. The performance–trust continuum. Five placement points span resident in-process execution (maximum authority, maximum required trust) to external E3 processes (maximum isolation and language freedom). The in-process contracts are portable C ABIs; the E3 end is stack-agnostic.

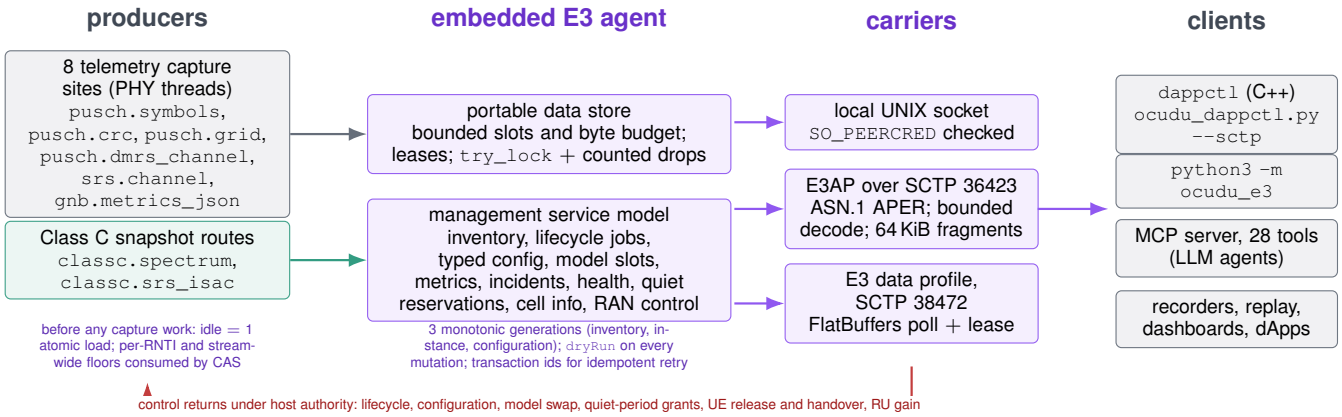


Fig. 5. The E3 plane. Producers pay 1 atomic load per event when nobody listens and consume rate floors by CAS before any capture work. One embedded agent with a bounded store serves 3 carriers; control returns under host authority.

TABLE I
TELEMETRY STREAMS (PUSH SUBSCRIPTIONS AND PULL POLLS).

Stream	Content
`pusch.symbols`	Equalized symbols + SINR, EVM, EPRE, RSRP, TA, CFO + grant shape
`pusch.crc`	Per-transport-block CRC verdict
`srs.channel`	Per-subcarrier LS SRS response per rx/tx port pair
`pusch.dmrs_channel`	DM-RS channel estimate per port and layer
`gnb.metrics_json`	The gNB JSON metrics report, byte for byte
`classc.spectrum`	Class C snapshot: full uplink grid tensor
`classc.srs_isac`	Class C snapshot: SRS channel-estimate tensor
`pusch.grid`	Raw pre-EQ REs + all parameters for offline re-decode

release, published to gather feedback, new use cases, and vetting by others, with upstreaming into the OCUDU mainline as the goal. Table II maps each mechanism in this paper to its entry point, so a reader can go from a sentence here to the code that implements it and, where warranted, to the working-group discussion that will shape it.

`ocudu-dapp-platform` is the gNB with the embedded runtime. The public ABI is `include/ocudu/dapp/abi/v1/` (module, host API, memory, execution, management, metric catalog), the 6 use-case interfaces are `include/ocudu/dapp/use_cases/v1/` together with `coordination.h` for the quiet-reservation intent, and the out-of-process contracts (shared ring layout, CUDA export pool, SEQPACKET control protocol) are `include/ocudu/dapp/process/v1/`. The runtime under `lib/dapp/runtime/` holds the package catalog and manifest, the artifact verifier and signature checks, the disposable preflight, the module loader, the native hook registry and slots, the instance manager and lifecycle, the job dispatcher, the circuit breaker, the quiescence tracker, and the Class C process supervisor. The E3 plane under `lib/dapp/e3/` holds the embedded agent, the management service, the APER and FlatBuffers codecs, the E3AP and E3DP SCTP servers, the local credentialed transport, the portable data store, and the telemetry publisher; `lib/`

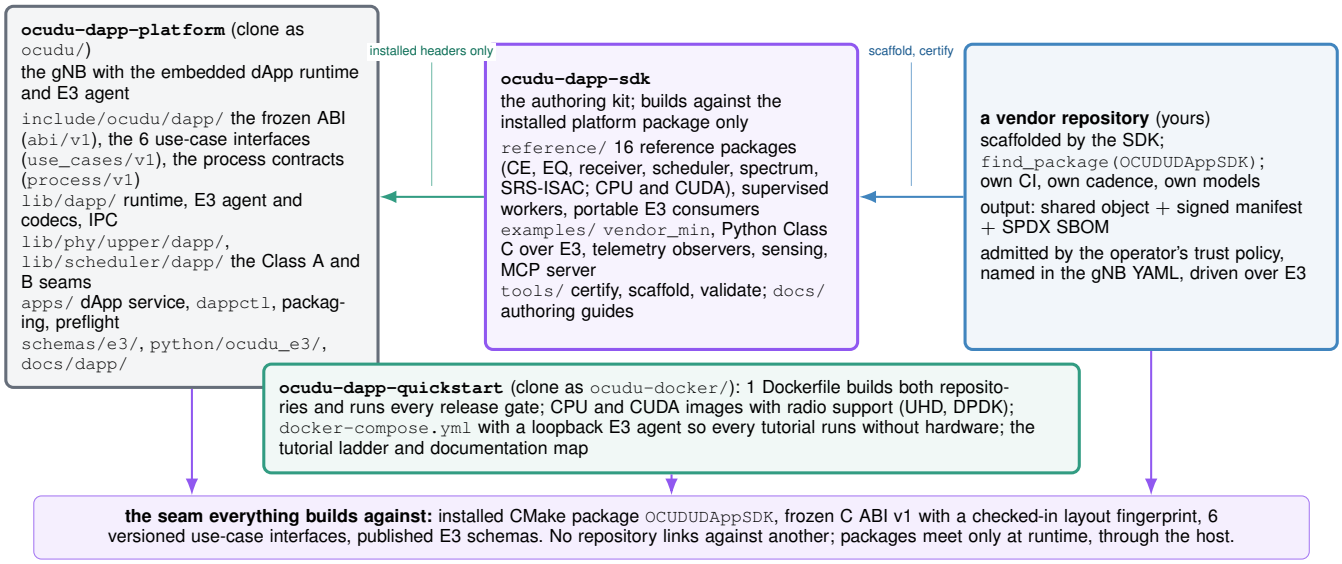


Fig. 6. The 3 public repositories and the seam between them. The platform repository owns the interfaces, the runtime, and the E3 plane; the SDK and every vendor repository build against the installed package alone; the quickstart builds all of it into a container with a loopback E3 agent.

`dapp/ipc/` holds the shared SPSC ring, the SEQPACKET channel, and the Class C publication routes. The seams into the PHY and MAC are `lib/phy/upper/dapp/` and `lib/scheduler/dapp/`. The gNB-side service and YAML configuration are `apps/services/dapp/`; the operator tools `dappctl`, `dapp_package`, and `dapp_preflight` are under `apps/tools/`; the schemas are `schemas/e3/`; the Python client is `python/ocudu_e3/`; and `docs/dapp/` holds the 13 tutorials and the reference documents.

`ocudu-dapp-sdk` is the authoring kit. It consumes the installed `OCUDUDAppSDK` CMake package and never copies host headers. `reference/` holds 16 reference packages (channel estimator, equalizer, receiver, scheduler, spectrum, SRS-ISAC, in CPU and CUDA variants), the supervised `process/` workers, and the `portable/` E3 consumers; `examples/` holds the copy-me commercial Class A layout `vendor_min`, a Python Class C dApp over E3, Python telemetry observers, spectrum and ISAC sensing over E3AP, and the MCP server; `tools/` holds the certifier, the scaffolds for every class, and the validation ladder; `docs/` holds the per-class authoring guides.

`ocudu-dapp-quickstart` is 1 Dockerfile that builds both repositories from their current commits, runs every release gate inside the build, and produces CPU and CUDA images with radio support (UHD, DPDK) baked in, plus a Compose file with a loopback E3 agent so every tutorial command runs without hardware.

VII. Working With the Platform

The first session. The zero-hardware path builds the platform into a container and points every later command at a loopback E3 agent; a green image is a validated platform, because every release gate runs inside the build.

```
# 1. Clone the three public repositories side by side
WG2=https://gitlab.com/ocudu/work_groups/wg2_ai_ran
git clone $WG2/ocudu-dapp-platform.git ocudu
git clone $WG2/ocudu-dapp-sdk.git
git clone $WG2/ocudu-dapp-quickstart.git ocudu-docker
# 2. Build the release images (host, SDK, every gate)
ocudu-docker/scripts/build.sh
# 3. Start the loopback E3 agent and look around
docker compose -f ocudu-docker/docker-compose.yml up e3-
    agent
python3 -m ocudu_e3 --port 36423 catalog
python3 -m ocudu_e3 --port 36423 subscribe pusch.crc --
    maximum 5
python3 -m ocudu_e3 --port 36423 inventory
```

The tutorial ladder. The documentation is organized as 13 tutorials (Table III) that take a reader from an empty vendor repository to an LLM agent tuning a live cell; the first 5 need no signing keys, and all of them run against the loopback agent.

Building a dApp. A vendor scaffolds a project with `scaffold_class_a_cuda.py` or `scaffold_class_bc_cpu.py`, builds it with 1 `find_package(OCUDUDAppSDK)` against the installed package, and runs `certify_class_a_cuda` (for Class A) or `dapp_validate.py`: the certifier loads the final shared object through the public ABI, runs the complete lifecycle, creates 2 workers on independent CUDA streams, and drives deterministic vectors from the 1-PRB minimum to the package's declared maximum, including off-origin placements that catch a module interpreting DM-RS coordinates relative to its allocation and rank-deficient 2-layer references that resolve only when the joint multi-symbol pilot set is consumed; every caller-owned output sits at a nonzero offset inside a poisoned allocation with prefix, suffix, and inactive-capacity canaries checked after every call. `dapp_package` then emits the manifest and SBOM and signs the bundle with the vendor's key; the operator's trust policy names that key, with epochs, revocation, and per-package sequence floors.

TABLE II
WHERE EACH MECHANISM LIVES (PATHS RELATIVE TO THE PLATFORM REPOSITORY UNLESS MARKED SDK).

Mechanism	Entry point
Frozen C ABI v1	`include/ocudu/dapp/abi/v1/module.h`, `host_api.h`, `memory.h`, `execution.h`; fingerprint gate in the SDK CI
Use-case interfaces	`include/ocudu/dapp/use_cases/v1/` (`spectrum.h`, `srs_isac.h`, `channel_estimator.h`, `equalizer.h`, `receiver.h`, `scheduler.h`, `coordination.h`)
Class A seam	`lib/phy/upper/dapp/`; hook slots in `lib/dapp/runtime/native_hook_*.cpp`
Class B seam	`lib/scheduler/dapp/`; intent validation and allow flags in the scheduler policy
Class C native and process	`lib/dapp/ipc/publication_route.cpp`, `shared_spsc_ring.cpp`, `seqpacket_channel.cpp`; `lib/dapp/runtime/class_c_process_supervisor.cpp`
Admission and trust	`lib/dapp/runtime/package_catalog.cpp`, `package_manifest.cpp`, `package_signature.cpp`, `artifact_verifier.cpp`, `native_preflight.cpp`
Lifecycle and jobs	`lib/dapp/runtime/instance_manager*.cpp`, `lifecycle.cpp`, `job_dispatcher.cpp`, `quiescence_tracker.cpp`
Safety	`circuit_breaker.cpp`, `guarded_call.h`, `incident_store.cpp`
E3 agent and codecs	`lib/dapp/e3/embedded_agent.cpp`, `management_service.cpp`, `aper_codec.cpp`, `e3ap_codec.cpp`, `fbs_codec.cpp`
Carriers	`lib/dapp/e3/local_connector.cpp`, `e3ap_sctp_server.cpp`, `e3dp_sctp_server.cpp`
Telemetry	`lib/dapp/e3/telemetry_publisher.cpp`, `portable_data_store.cpp`
Schemas	`schemas/e3/` (ASN.1 management, telemetry, E3AP; FlatBuffers data profile)
YAML and service	`apps/services/dapp/`; reference in `docs/dapp/yaml_configuration.md`
Tools	`apps/tools/dappctl`, `dapp_package`, `dapp_preflight`; `python/ocudu_e3/`
References (SDK)	`reference/`: `channel_estimator`, `equalizer`, `receiver`, `scheduler`, `spectrum`, `srs_isac` (each with a `_cuda` variant), `process`, `portable`
Certify and scaffold (SDK)	`tools/`: `certify_class_a_cuda.cu`, `scaffold_class_a_cuda.py`, `scaffold_class_bc_cpu.py`, `dapp_validate.py`
MCP server (SDK)	`examples/e3_mcp_server/`; guide in `docs/mcp_companion.md`

Loading and driving a dApp. The operator stages the signed bundle in a catalog directory named in the gNB YAML (`dapp:` block; `initial_instances` for instances created at startup) and drives it over E3:

```
# stage a bundle, then walk the lifecycle over the local
    socket
ocudu-dappctl inventory
ocudu-dappctl load --package <id>            # verified,
    preflighted
ocudu-dappctl lifecycle prepare  <instance>  # allocate,
    warm
ocudu-dappctl lifecycle arm      <instance>  # attach hooks,
     not yet active
ocudu-dappctl lifecycle shadow   <instance>  # invoked,
    measured, never applied
ocudu-dappctl lifecycle activate <instance>  # atomic slot
    exchange
ocudu-dappctl config  <instance> --set key=value --dry-run
ocudu-dappctl model   <instance> stage|validate|warm|
    activate|rollback
ocudu-dappctl metrics <instance>;  ocudu-dappctl incidents;
     ocudu-dappctl health
# the same operations over SCTP from any host
python3 ocudu_dappctl.py --sctp --host <gnb> inventory
```

TABLE III
THE TUTORIAL LADDER (`docs/dapp/` IN THE PLATFORM, `docs/` IN THE SDK).

#	Tutorial	Covers
0	Zero-hardware testbed	build the images; loopback E3 agent; run everything radio-free
1	New vendor dApp repo (SDK)	scaffold, build, certify against the installed SDK
2	Install and load from the gNB YAML	catalog, trust policy, initial instances, admission
3	Operate with `dappctl`	lifecycle, A/B version swap, deactivate and unload, breakers
4	Monitor and tune over MCP (SDK)	ask the gNB questions; KPI-driven LLM tuning loops
5	Signing, SBOMs, keys	what is signed, the SBOM, trust policy, rotation, revocation
6	Exposing dApp KPIs over E3	declare metrics; read them from any client
7	First dApp: standalone Class C	a Python out-of-process dApp: subscribe, decode, act back
8	Telemetry and data capture	all 8 streams, floors, HDF5 capture, offline PUSCH replay
9	Neural-model operations	stage, validate, warm, activate, rollback; live A/B
10	Class B scheduler control	the 100 μs contract, allow flags, evaluation-first, sensing to avoidance
11	Troubleshooting runbook	symptom, cause, fix, from real failure modes
12	Capstone	capture, train, package, shadow, activate, monitor, rollback

Shadow state deserves emphasis: an instance can receive configuration and model operations before activation, so weights are staged, validated, and warmed against live traffic shape with 0 exposure, then activated atomically under a generation check, and rolled back the same way. Two instances of competing packages can be loaded, 1 in shadow and 1 active, and swapped by lifecycle alone; this is how equalizer variants were compared over the air (Section XI).

Writing a portable Class C dApp. A Python process on any host opens an E3AP association, subscribes to `classc.spectrum` or `srs.channel`, decodes each indication into a NumPy array, and acts back through management operations; the SDK's `class_c_e3_python` example and `e3_spectrum_sensing` do exactly this and were active in the composition run of Section XI. Nothing in the gNB is patched or rebuilt.

Driving the cell from an agent. The MCP server in the SDK (`examples/e3_mcp_server/`) is an E3 client that exposes the interface's KPIs and knobs to LLM agents in the simplest possible form: 28 Model Context Protocol tools over 1 E3 association, 20 that observe (sector state, per-UE KPIs, traffic history, SRS channel, constellation, telemetry catalog, dApp inventory, health, metrics, configuration, incidents, jobs, quiet reservations) and 8 that act (dApp configuration and model operations, lifecycle, UE release and handover, RF gain). An agent tunes and controls the cell by calling tools, with no protocol knowledge; every state-changing tool dry-runs by default, and the platform's guardrails (typed validation, generations, allow flags, host-arbitrated RAN control) bound every call.

VIII. Seven Use Cases on One Architecture

Each use case below is stated as what runs, which path it exercises, and what the platform provides; Table V records what actually ran and with what evidence.

Spectrum sensing, the direct descendant of the original dApp demo, at 3 escalating trust levels: a Python E3 observer on the Class C spectrum stream, functionally the Northeastern demo over published schemas with rate floors and no patched gNB; the same radiometer as a supervised process dApp (`ref_spectrumd`) under heartbeats and seccomp; and the worker as *authority*, returning a per-PRB interference map and quiet-period intents that appear in the scheduler's avoidance mask within a few slots. One algorithm, 3 containment levels, no DU change.

ISAC rides the SRS channel the gNB already estimates: `srs_isac_monitor.py` sketches a range–Doppler map from `srs.channel`; the packaged Class C references compute a bounded channel-impulse-response preview from the same tensors, the CUDA variant on the producer's stream without ever materializing it on the host.

Dataset capture is a first-class product. `record_telemetry.py` writes any stream mix to HDF5; `export_pusch_burst.py` captures `pusch.grid` bursts; `ocudu-pusch-replay` re-decodes them offline with the gNB's own receiver and cross-checks the recorded CRC verdict. Every capture carries its own verified label, channel estimates, and exact PHY configuration.

The continual-learning loop (Fig. 7) assembles the above: telemetry streams into an online tuning process, a dApp, a RIC application, or an edge job, whose sealed model artifact returns over the same E3 surface, staged, validated, and warmed on a shadow instance against live traffic shape with 0 exposure, then activated atomically under a generation check. Per-model metrics and incidents monitor it; degradation triggers 1-operation rollback; drift re-enters at capture. Every step is a typed, transaction-id'd, dry-runnable operation, so the loop can be driven by a script, a CI pipeline, or an agent.

The inline neural receiver is the flagship Class A application. The SDK's CUDA references implement the exact contracts a neural receiver adopts at the 3 depths, with the model machinery a learned implementation needs: prepare-time weight upload, per-worker lane-local history, 2 bounded device model slots, and completion-fence discipline so an inactive slot is never freed under a reading stream. The neural equalizer in operation on the cell is *not* in the SDK: it is a neural equalizer package built out of tree, against the public SDK alone, by a separate team, which is itself the ecosystem claim demonstrated.

The inline learned scheduler. A learned policy replaces stages behind the same intent schema as the baseline replica and inherits the same evaluation-first path, the deadline, the post-equalization noise from Class A, and the interference context from Class C, so it can be interference-aware on day 1.

Agent-driven operations. Because every operation is a typed, schema'd request, exposing them to an LLM agent is a thin adapter: the MCP server turns the E3 interface's KPIs and knobs into tools an agent can read and turn. An agent can run a standing loop, read KPIs and incidents every few minutes, reason about trends, and adjust cell parameters or dApp configuration from a single natural-language prompt, with the same guardrails that make the loop safe for a script.

IX. The Vendor Ecosystem: Build, Load, Compose

Everything in Section VIII was 1 team's applications. The same infrastructure supports teams that have never met (Fig. 8), through 1 clean seam: the host repository owns the 6 interfaces, the E3 plane, and the runtime; the installed SDK package, the frozen ABI, and the published schemas are the only things anyone builds against; every producer of dApps sits above that seam with the same headers, certification tools, and packaging.

Vendors: your repository, your cadence. A dApp lives in the vendor's own repository, built against the installed SDK with 1 `find_package`: no host source tree, no fork, no patch queue. The fingerprint-verified ABI and the published compatibility policy decouple the vendor's release cadence from the host's; offline certification and the validation ladder give the vendor a proof to ship beside the artifact, an immutable, hash-bound bundle of shared object, manifest, SPDX SBOM, and detached signature.

Operators and integrators: compose, tune, optimize. The operator holds the other half of the trust relationship: an offline trust policy naming each vendor's keys, with epochs, revocation, and per-package sequence floors. Composition is a catalog decision, signed bundles from several vendors named in YAML and driven over E3, and it is safe by construction because packages cannot link against or call each other, cooperate only through host-owned caches, and every authority is individually granted. The operator therefore tunes the combination: allow flags decide how much scheduling authority a policy gets, budgets and floors decide how much real-time and telemetry cost a sensing suite may impose, placement decides its containment, and shadow-first lifecycles with per-instance metrics make A/B between competing packages routine.

A vendor-neutral home, and a path to standards. The code lives under the OCUDU AI-RAN Working Group 2 as a preview release, and the working group is the venue in which it is being vetted, extended with new use cases, and prepared for upstreaming into the OCUDU mainline. Interfaces defined in code, exercised by shipping references, and stress-tested by vendors interworking through them let technical consensus form in the project, at the speed of working software; what then goes to the SDOs, O-RAN's E3 work [11] and ultimately 3GPP, is a proven artifact. The Class A and Class B C ABIs are deliberately portable beyond this codebase, and the E3 end already admits any stack that speaks the schemas.

X. The Latency Architecture, and Safety

A slot is 500 μs at 30 kHz subcarrier spacing, and at 5-cell CUDA L1 scale roughly 100 μs of host jitter already produces fronthaul lateness, so the platform must add nothing

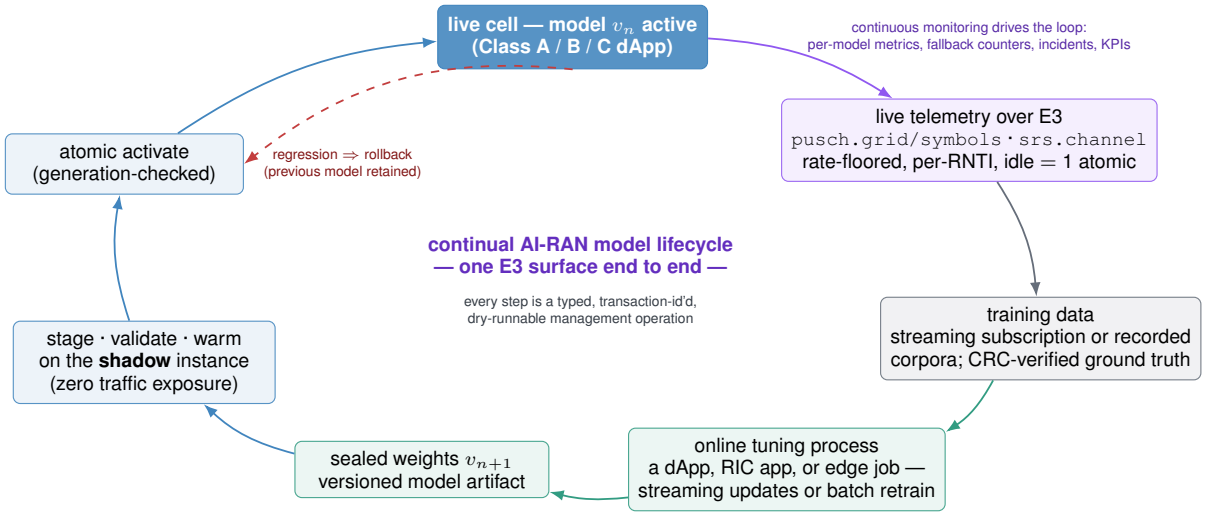


Fig. 7. The continual-learning loop. Rate-floored telemetry streams into an online tuning process; sealed model artifacts return over E3, staged, warmed on a shadow instance, and activated atomically; monitoring drives rollback or the next round. One E3 surface carries the whole loop.

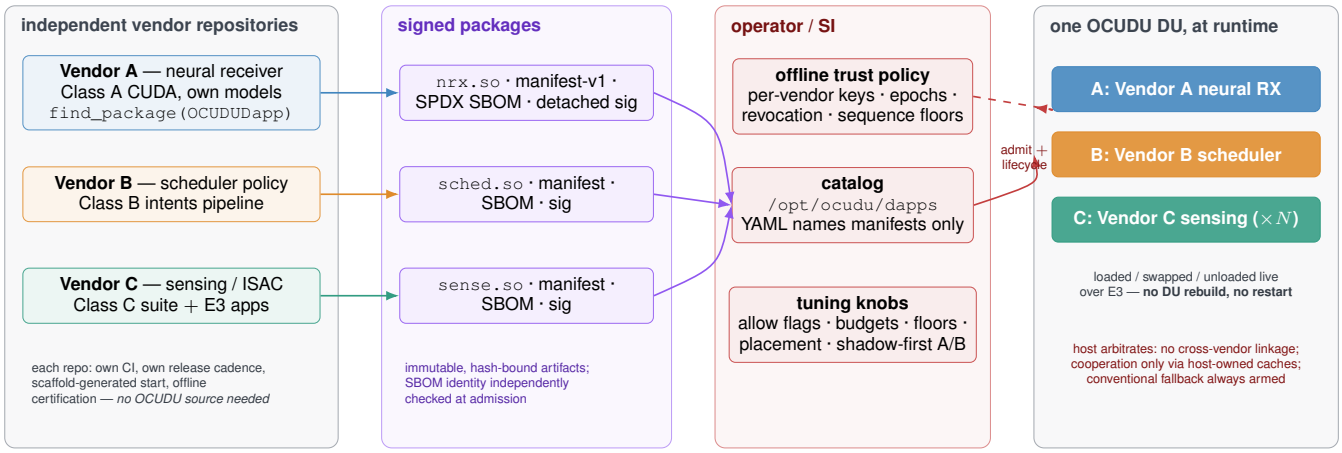


Fig. 8. The ecosystem the infrastructure enables. Vendors build signed packages in their own repositories against the installed SDK; operators and integrators hold the trust policy, compose packages from several vendors in 1 catalog, and load, tune, swap, and unload them on a live DU: no rebuild, no cross-vendor linkage, host arbitration throughout.

measurable to the radio's steady state. The design guarantees this by construction (Fig. 9) in 2 moves. *First*, every steady-state mechanism on a real-time thread costs nanoseconds to single-digit microseconds: an atomic gate when idle, 1 CAS for a rate floor, a lock-free cache read, a try-lock publication with 1 eventfd write. *Second*, work is placed by thread, not by module: the real-time lane's entire E3 duty is 1 gated memcpy, while encoding, socket I/O, and lifecycle jobs live on management executors that are allowed to be slow. Five rules make this concrete:

1) **Idle = 1 atomic load.** Every capture site and dApp hook is gated by a single relaxed/acquire atomic; an unconfigured feature has no measurable cost.
2) **Producers never block.** Try-locks, non-blocking eventfds, and counted drops replace every wait a consumer could induce; a slow observer is its own problem, never the radio's.
3) **Every residual wait is bounded.** A degraded GPU or wedged consumer costs at most the current slot: fallback fires, a counter increments, no thread stalls.
4) **No steady-state allocation** on any invocation path: pre-allocated workspaces, lock-free snapshot pools, builders cleared per encode.
5) **GPU discipline:** Class A enqueues on the host's stream and returns; Class C CUDA export is 1 device-to-device snapshot into a bounded IPC pool, never a mapping of live L1 memory.

Fig. 10 shows the ladder a package climbs and the rail everything falls back to. Admission binds the artifact hash and SBOM identity, runs descriptor discovery in a disposable preflight process, and checks declared shape profiles before any dispatch. Runtime guards catch exceptions at the module boundary and translate them into sanitized incident records, meter deadlines on separate callback and comple-

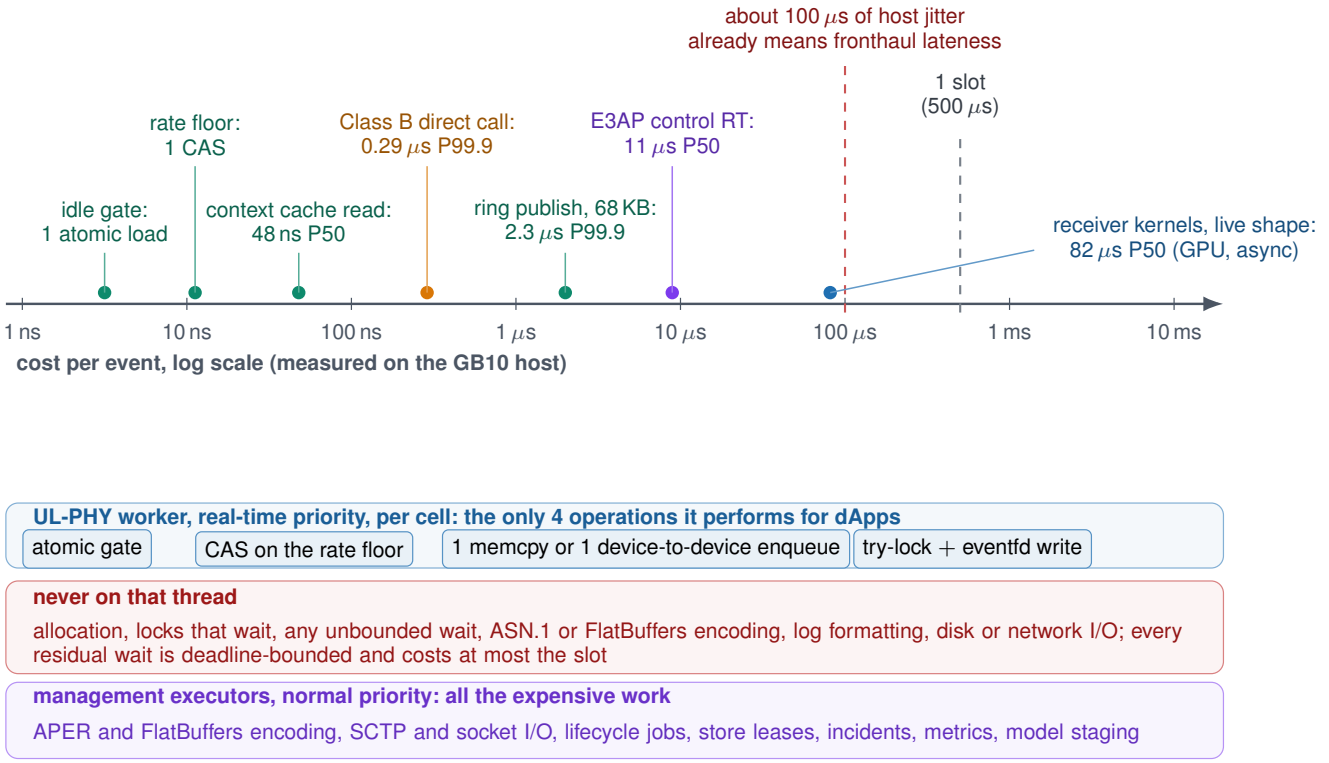


Fig. 9. The latency architecture. Top: every steady-state mechanism on the real-time path on a log cost axis, 3 orders of magnitude inside the slot budget and far below the jitter that already causes fronthaul lateness. Bottom: work placement by thread. The real-time lane owns 4 cheap operations; everything expensive runs at normal priority.

tion clocks, and open per-lane circuit breakers on faults, with the conventional path armed at every rung. Supervised Class C workers add authenticated `SOCK_SEQPACKET` control with pinned peer credentials, pointer-free results validated at the trust boundary, seccomp deny-lists, resource limits, challenge–response heartbeats, and a backed-off, healthy-uptime-windowed restart budget. Observability closes the loop: per-reason fallback counters, deadline misses with measured GPU durations, heartbeat latencies, restart and incident records, all queryable over the same E3 surface a dApp is managed through.

XI. Measured Evidence and Limitations

Testbed. All live-cell results come from 1 over-the-air cell: an NVIDIA GB10 (DGX Spark, Arm) host running the CUDA-resident L1, a split-8 USRP B210, band n78 at 3410.1 MHz, 20 MHz (51 PRB) TDD at 30 kHz subcarrier spacing, 1 transmit and 1 receive antenna, an Open5GS core, and commercial handsets as UEs. Interface measurements come from the companion interface study [20], taken on the same host with the released mechanisms and raw percentiles under 2 conditions, a quiet host and the cell on the air (Table IV, Figs. 11 and 12); the benchmark suite that produced them is published beside the platform.

Limitations and status. *Class A tails:* the reference receiver meets its deadline at the live shape on an exclusive GPU, but the 273-PRB kernels complete in about 270 μs against a 150 μs deadline and are not yet latency-qualified; that qualification is the next admission gate. *Class B evidence* was gathered on a host without approved CPU controls and is development evidence; the suite shows that on a quiet host every carrier meets the deadline and that only the direct call keeps its tail once the DU is on the air, which is the condition that matters. *Steady-state overhead* is guaranteed by construction and checked by fronthaul-lateness release gates, but a published disabled/enabled/enabled-with-recorder comparison is still pending. *Ecosystem:* the multi-vendor machinery is implemented and exercised, yet every package that has run on the cell so far was produced by the authors' organization; the out-of-tree neural equalizer demonstrates the seam, not yet an independent vendor. *Transport security:* E3 carries no transport authentication in this release; the credentialed local socket, a loopback bind, and operator tunnelling are the boundary. *MCP:* the server is an example of the pattern, not a hardened product; its safety comes from the platform's guardrails. *Multi-cell* telemetry attribution and an asynchronous Class C prepared-output transaction remain roadmap items.

XII. What the Expanded Contract Buys

Table VI summarizes the delta against the pioneering frameworks of Section II, and it should be read as a statement of scope and production maturity: those frameworks set out to prove the tier and succeeded, and much of what follows is machinery one needs only once dApps are trusted enough

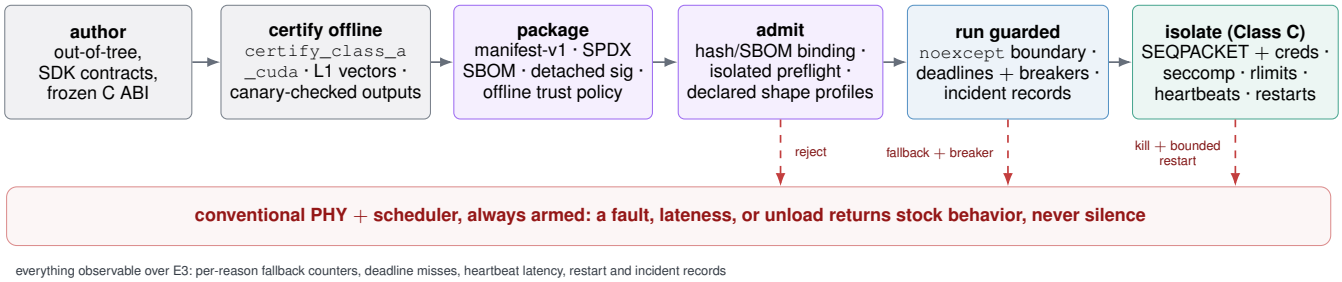


Fig. 10. The safety ladder. Certification and signing happen offline; admission is fail-closed; runtime guards convert faults and lateness into fallback plus breaker; supervised isolation bounds even a misbehaving process. The conventional path is armed at every rung, and every transition is observable over E3.

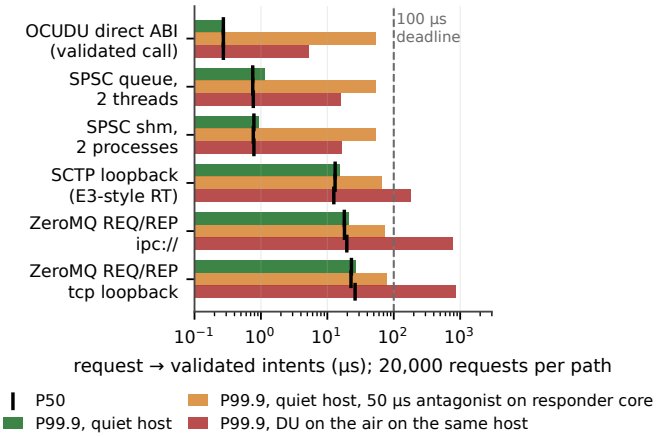


Fig. 11. The Class B contract under load. One 3.2 KB scheduler request over 6 boundaries, 20,000 requests each; bars are P99.9, ticks P50, the dashed line the admitted deadline. On a quiet host every carrier fits; with the DU on the air only the paths without a wake keep their tails.

TABLE IV
MEASURED EVIDENCE (GB10 HOST). RUNTIME ROWS ARE THE PLATFORM'S OWN CHECKPOINTS; SUITE ROWS ARE FROM THE COMPANION INTERFACE STUDY [20], QUIET HOST UNLESS STATED.

Claim	Result	Conditions
Class B boundary (runtime)	3.6 μs P99.9, 0 misses	worst profile, 9,000-call matrix
Class B direct call (suite)	0.29 μs P99.9 quiet; 5.2 μs with the cell on air	same 3.2 KB request over SCTP loopback: 16 μs quiet, 181 μs on air, 0.35 % misses at 100 μs
Class A resident receiver	82 μs P50, 112 μs P99.9 (suite); 92.5 / 105.1 μs (runtime)	live shape, 51 PRB, 2 layers, 256-QAM, exclusive GPU; 150 μs deadline; 273-PRB kernels not yet qualified
Class C ring publication	2.0 μs P50, 2.3 μs P99.9 per 68 KB slot	producer critical section; full ring is a counted drop; 0.87 Gb/s sustained across forced worker restarts (runtime)
E3AP over SCTP	10 μs one way at 1.5 KB; 1.4 ms at 734 KB; 11 μs control round trip	real asn1c APER codec, loopback association, 2 pinned processes
Composition on 1 cell	4 dApps, all 3 classes, 260,000+ equalizer invocations, 0 fallbacks	neural equalizer (A), scheduler (B), spectrum and SRS-ISAC (C); attached handsets
Equalizer variants over the air	7–11 percentage points lower BLER at MCS 11–15	2 neural variants swapped by lifecycle alone, same UEs and traffic
Operations surface	8 telemetry streams; 28 MCP tools (20 observe, 8 act)	1 E3 association; every state-changing tool dry-runs by default

to matter. Three rows carry the argument. *Authority*: prior control returns were coarse primitives; here every control path, Class A outputs, Class B intents, Class C maps and quiet requests, E3 RAN control, passes typed validation against operator-granted bounds, and the host remains the oracle. *Producer safety*: prior exports were coupled to L1; here the radio is arithmetically indifferent to its observers. *Operations*: prior dApps were hand-integrated deployments; these are governed artifacts with signatures, SBOMs, generations, dry-runs, incidents, and rollback, which is what makes multi-vendor composition and agent-driven operation possible at all.

XIII. Conclusion and Availability

Artifact availability. All software is open source under the BSD-3-Clause-Clear license in the OCUDU AI-RAN Working Group 2 group, https://gitlab.com/ocudu/work_groups/wg2_ai_ran/, as 3 repositories: `ocudu-dapp-platform` (the gNB with the dApp runtime, E3 agent, schemas, and Python client), `ocudu-dapp-sdk` (reference packages, certifier, scaffolds, examples, and the MCP server), and `ocudu-dapp-quickstart` (the reproducible container build, the loopback E3 agent, and the index of tutorials and documentation). The listing in Section VII reproduces the E3 surface with no radio, GPU, or signing keys; the over-the-air results require a GB10-class host, a USRP or split-7.2 O-RU, and attached UEs, as the tutorials describe. This is a preview release: the working group invites feedback on the interfaces, new use cases that stress them, and independent vetting of the references, all of which feed the upstreaming of the runtime and E3 plane into the OCUDU mainline.

Outlook. The ABI and schemas are frozen; the roadmap adds an asynchronous Class C prepared-output transaction, multi-cell telemetry attribution, and a broadening Class B feature block as evaluation-mode evidence accumulates. The platform is built to keep growing: new use-case interfaces, new streams, new references, and, by design, vendor dApps the authors will never see, composed by operators the vendors

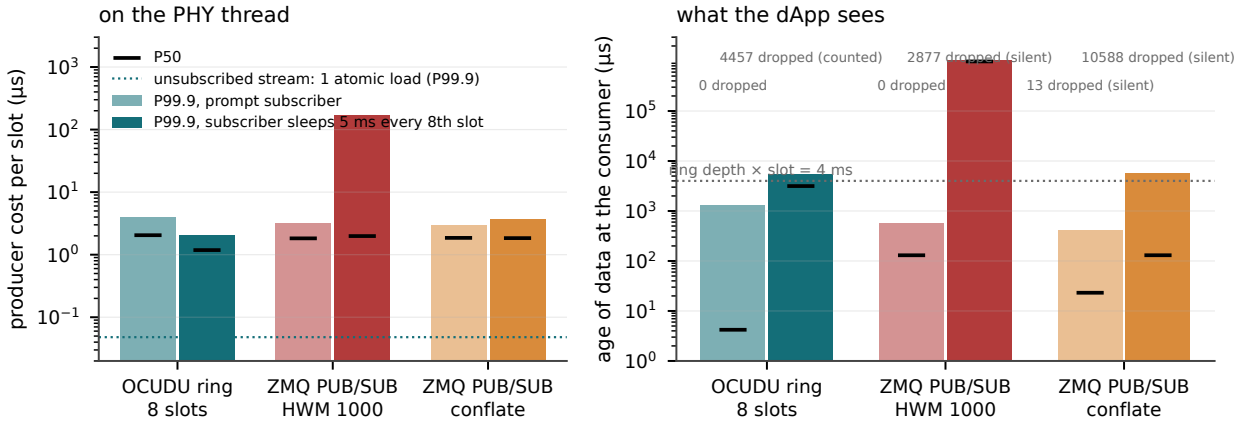


Fig. 12. The producer rule, measured. Per-slot producer cost (left) and the age of the data the consumer sees (right) for the OCUDU shared ring and ZeroMQ publish/subscribe, with a prompt subscriber and one that sleeps 5 ms every 8th slot. The ring's producer cost does not move, staleness is bounded by ring depth, and every drop is counted; ZeroMQ queues to its high-water mark or drops silently.

TABLE V
WHAT RAN FOR EACH USE CASE, AND THE EVIDENCE BEHIND IT.

Use case	What ran	Evidence
Spectrum sensing	Python E3 observer; `ref_spectrumd` supervised worker; spectrum authority	live cell, active in the composition run
ISAC	`srs_isac_monitor.py`; `ref_srs_isac_cuda`	live cell, active in the composition run
Dataset capture	`record_telemetry.py`, `export_pusch_burst.py`, `ocudu-pusch-replay`	live captures re-decoded offline; recorded CRC verdicts reproduced
Continual-learning loop	stage, validate, warm, activate, rollback on the receiver reference	packaged end-to-end campaigns on a dummy-RU DU
Inline neural receiver	SDK CUDA references (conventional kernels); out-of-tree neural equalizer	live cell, resident; over-the-air variant comparison
Learned scheduler	`ref_scheduler_cpu` in evaluation mode, then per-flag authority	live cell, active in the composition run
Agent operations	28-tool MCP server	interactive sessions against the live cell

TABLE VI
PRIOR DAPP FRAMEWORKS (BUILT TO PROVE THE TIER) VERSUS THE OCUDU PLATFORM (BUILT FOR PRODUCTION ADOPTION). THE DELTA IS SCOPE AND MATURITY, NOT INTENT.

	Prior (NEU / NVIDIA)	OCUDU dApp platform
Execution	External process over exported IQ/KPI	External *and* in-process; resident GPU L1 replacement; bounded scheduler hook
Control	Early primitives (e.g., PRB blacklist)	Typed, validated, allow-flag-bounded intents; granted quiet reservations; RAN control ops
Producer safety	Export coupled to L1	Never blocks; idle = 1 atomic; bounded waits; counted drops
Fallback	Left to the integration	Conventional path always armed; breakers; quarantine
Packaging	Per-deployment integration	Signed manifest + SBOM + offline trust; preflight; ABI fingerprint
Lifecycle	Manual or scripted	E3 jobs, shadow state, generations, dry-run, rollback
Model ops	External to the framework	Stage, validate, warm, activate, rollback on a live cell
Multi-vendor	Vendor-neutral in intent; composition machinery still ahead	Runtime composition under operator trust and allow flags
Isolation	Container	Supervised process: authenticated IPC, seccomp, heartbeats, bounded restart
Wire	Pre-standard, still converging	Published ASN.1 APER + FlatBuffers, bounded codecs, versioned schemas; SCTP as first-class carrier

will never meet. Because the interfaces live in a vendor-neutral open-source project, the consensus they accumulate is portable into O-RAN and 3GPP when the time comes to formalize them.

The dApp tier's founders were right about where the opportunity lives, and the observer loop was the correct first step into it. This paper has presented the next step as a working system: 1 infrastructure on which the same governed artifact can observe the spectrum today, advise the scheduler tomorrow, and replace the receiver the day the evidence supports it, while feeding the datasets that train its own successor, answering to an operations agent, and sharing the cell with dApps from vendors who have never seen its code.

ACKNOWLEDGMENT

This work was supported in part by the U.S. DoD OUSD(R&E) FutureG Office through the National Spectrum Consortium (NSC) Spectrum Forward OTA, including support for contributions to the Linux Foundation's OCUDU open-source RAN project.

REFERENCES

[1] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 2, pp. 1376–1411, 2023.

[2] O-RAN ALLIANCE, "WG3: Near-real-time RAN intelligent controller and E2 interface workgroup," 2026, official workgroup charter. [Online]. Available: https://www.o-ran.org/technical-groups/wg3

[3] R. Wiesmayr, C. Dick, J. Hoydis, and S. Cammerer, "Design of a standard-compliant real-time neural receiver for 5G NR," *arXiv preprint arXiv:2409.02912*, 2024.

[4] S. D'Oro, M. Polese, L. Bonati, H. Cheng, and T. Melodia, "dApps: Distributed applications for real-time inference and control in O-RAN," *IEEE Communications Magazine*, 2022, arXiv:2203.02370. [Online]. Available: https://arxiv.org/abs/2203.02370

[5] A. Lacava, L. Bonati, N. Mohamadi, R. Gangula, F. Kaltenberger, P. Johari, S. D'Oro, F. Cuomo, M. Polese, and T. Melodia, "dApps: Enabling real-time AI-based open RAN control," *Computer Networks*, vol. 269, p. 111342, 2025, arXiv:2501.16502. [Online]. Available: https://arxiv.org/abs/2501.16502

[6] OpenRAN Gym and WiNES Lab, "dApp framework tutorial: Deploy and extend dApps," 2026, openAirInterface spectrum-sharing tutorial. [Online]. Available: https://openrangym.com/tutorials/dapps-oai

[7] WiNES Lab, "libe3: Vendor-neutral E3AP C++ library," GitHub repository, 2026. [Online]. Available: https://github.com/wineslab/libe3

[8] D. Villa, M. Belgiovine, N. Hedberg, M. Polese, C. Dick, and T. Melodia, "Programmable and GPU-accelerated edge inference for real-time ISAC on NVIDIA Aerial testbed," *arXiv preprint arXiv:2512.06493*, 2026. [Online]. Available: https://arxiv.org/abs/2512.06493v2

[9] NVIDIA, "Aerial CUDA-accelerated RAN," https://docs.nvidia.com/aerial/, 2024.

[10] K. Cohen-Arazi, M. Roe, Z. Hu, R. Chavan, A. Ptasznik, J. Lin, J. Morais, J. Boccuzzi, and T. Balercia, "NVIDIA AI Aerial: AI-native wireless communications," *arXiv preprint arXiv:2510.01533*, 2025. [Online]. Available: https://arxiv.org/abs/2510.01533

[11] O-RAN Alliance nGRG, "dApps for real-time RAN control: Use cases and requirements," O-RAN Alliance, Tech. Rep. RR-2024-10, Oct. 2024, contributed research report. [Online]. Available: https://www.o-ran.org/research-reports/dapps-for-real-time-ran-control-use-cases-and-requirements

[12] The Mobile Network, "When it comes to Nokia's new AI-RAN, keep your eye on the dApps," https://the-mobile-network.com/2026/07/when-it-comes-to-nokias-new-ai-ran-keep-your-eye-on-the-dapps/, Jul. 2026.

[13] C. Boeira, E. Baena, A. Lacava, T. Melodia, D. Koutsonikolas, and I. Haque, "Performance characterization of dApps in open radio access networks," arXiv preprint arXiv:2605.05426, 2026.

[14] N. N. Santhi, D. Villa, M. Polese, and T. Melodia, "InterfO-RAN: Real-time in-band cellular uplink interference detection with GPU-accelerated dApps," *arXiv preprint arXiv:2507.23177*, 2025. [Online]. Available: https://arxiv.org/abs/2507.23177

[15] M. Pennybacker, W. Liu, A. Kharchenko, and T. O'Shea, "GPU-resident CUDA acceleration for OCUDU 5G PHY and O-RAN fronthaul: Architecture and preliminary performance," arXiv preprint arXiv:2608.04338, 2026. [Online]. Available: https://arxiv.org/abs/2608.04338

[16] OCUDU Ecosystem Foundation, "OCUDU ecosystem foundation FAQ," https://ocudu.org/faq/, 2026, linux Foundation-hosted Open CU/DU ecosystem.

[17] The Open Group, "POSIX dlopen()." [Online]. Available: https://pubs.opengroup.org/onlinepubs/9699919799/functions/dlopen.html

[18] NVIDIA, "CUDA runtime API: Stream management," 2026. [Online]. Available: https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__STREAM.html

[19] ——, "CUDA programming guide: The CUDA driver API," 2026. [Online]. Available: https://docs.nvidia.com/cuda/cuda-programming-guide/03-advanced/driver-api.html

[20] T. O'Shea, M. Pennybacker, and A. Kharchenko, "Enabling AI-RAN applications through dApp architecture decisions: A measured study," 2026, companion paper, in preparation.